



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks

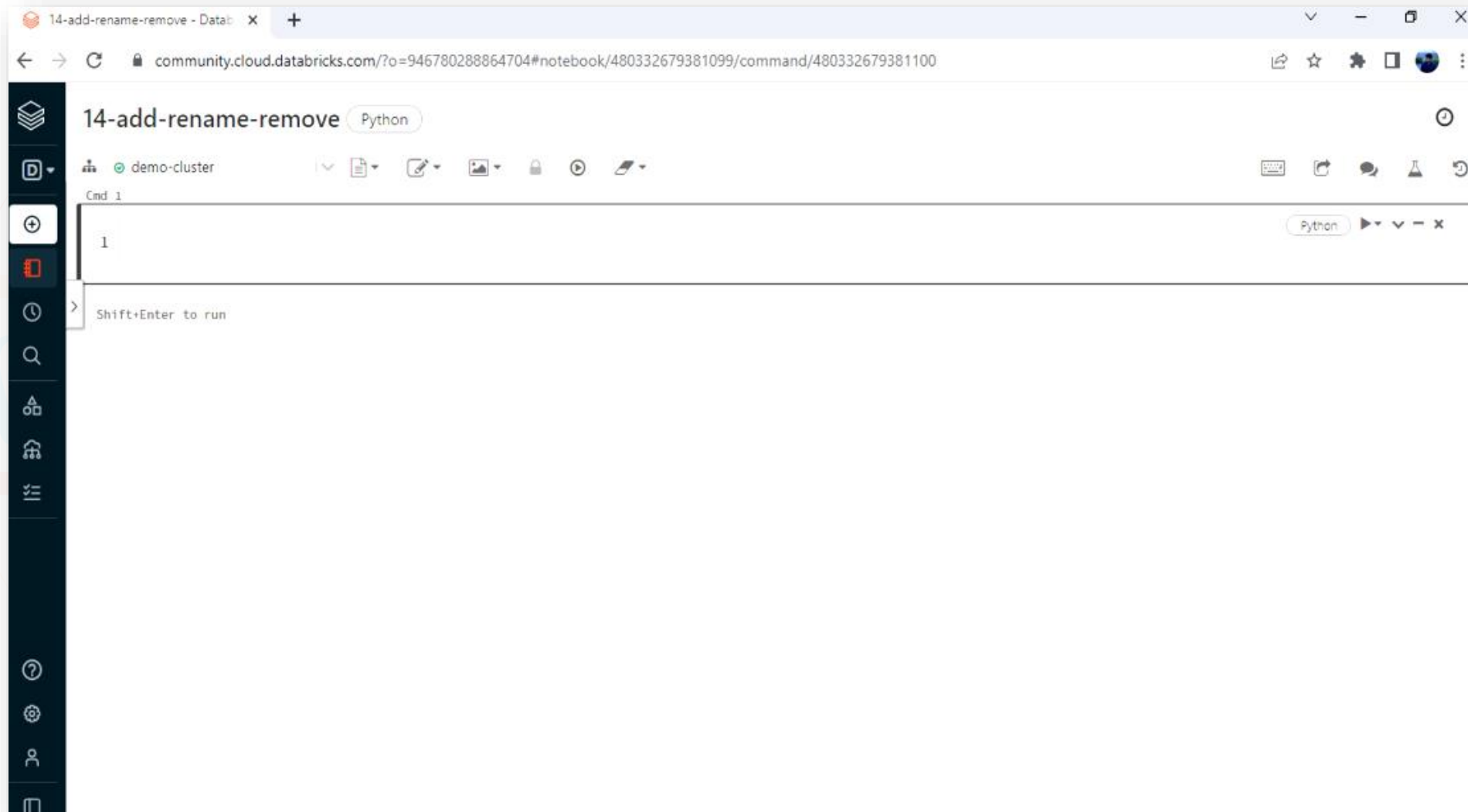


Adding, Remaining and Removing Columns

I want to talk about the following quickly in this chapter. The first thing that we want to cover is adding new columns to your dataframe. Spark dataframe gives you `withColumn()` transformation to do that. You can use `withColumn()` transformation to add a new column or transform an existing column. We have already used it in many places, and I hope you have already learned it. But let me give you a quick example once again.

1. Adding new columns to your data frame
 - `withColumn()`
2. Renaming Existing Columns in your data frame
 - `withColumnRenamed()`
3. Dropping columns from your data frame
 - `drop()`

Go to your Databricks workspace and create a new notebook (**Reference : 14-add-rename-remove**).



I am creating a data list and using it to define a data frame. You already learned this in an earlier lecture. Now I want to use a new column for this data frame. So I can use the `withColumn()` transformation.

14-add-rename-remove Python

demo-cluster

Cmd 1

```
1 data_list = [(100, "Prashant", 45, 45000),
2               (101, "Tarun", 36, 33000),
3               (102, "David", 48, 28000)]
4
5 test_df = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")
6 display(test_df)
```

(3) Spark Jobs

	id	name	age	salary
1	100	Prashant	45	45000
2	101	Tarun	36	33000
3	102	David	48	28000

Showing all 3 rows.

Command took 6.04 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:45:04 PM on demo-cluster

So you can use the `withColumn()` method to add a new field to your dataframe. The `withColumn()` method takes two arguments. The first argument is the column name and the second column is an expression to determine the value of the new columns. The `withColumn()` can add new columns or transform existing columns. The increment column doesn't exist in this dataframe, so the `withColumn()` will add it. However, if the column already exists, the `withColumn()` will simply transform it.

```
> Cmd 2  
1 from pyspark.sql.functions import expr  
2  
3 test_df1 = test_df.withColumn("increment", expr("salary * 10/100"))  
4 display(test_df1)
```

► (3) Spark Jobs

	id	name	age	salary	increment
1	100	Prashant	45	45000	4500
2	101	Tarun	36	33000	3300
3	102	David	48	28000	2800

Showing all 3 rows.

Command took 1.57 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:47:13 PM on demo-cluster

The salary column already existed, so the withColumn() updates the same column. The withColumn() is super convenient. However, it has got some problems. Each withColumn() method introduces a new projection in the SQL execution plan. Therefore, calling it multiple times can generate big SQL execution plans, which can cause performance issues and even StackOverflowException. To avoid this, we prefer to use the select() expression and avoid using the long list of withColumn() method.

```
> Cmd 3

1 test_df2 = test_df1.withColumn("salary", expr("salary + increment"))
2 display(test_df2)

▶ (3) Spark Jobs
```

	id	name	age	salary	increment
1	100	Prashant	45	49500	4500
2	101	Tarun	36	36300	3300
3	102	David	48	30800	2800

Showing all 3 rows.

Command took 1.30 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:49:07 PM on demo-cluster

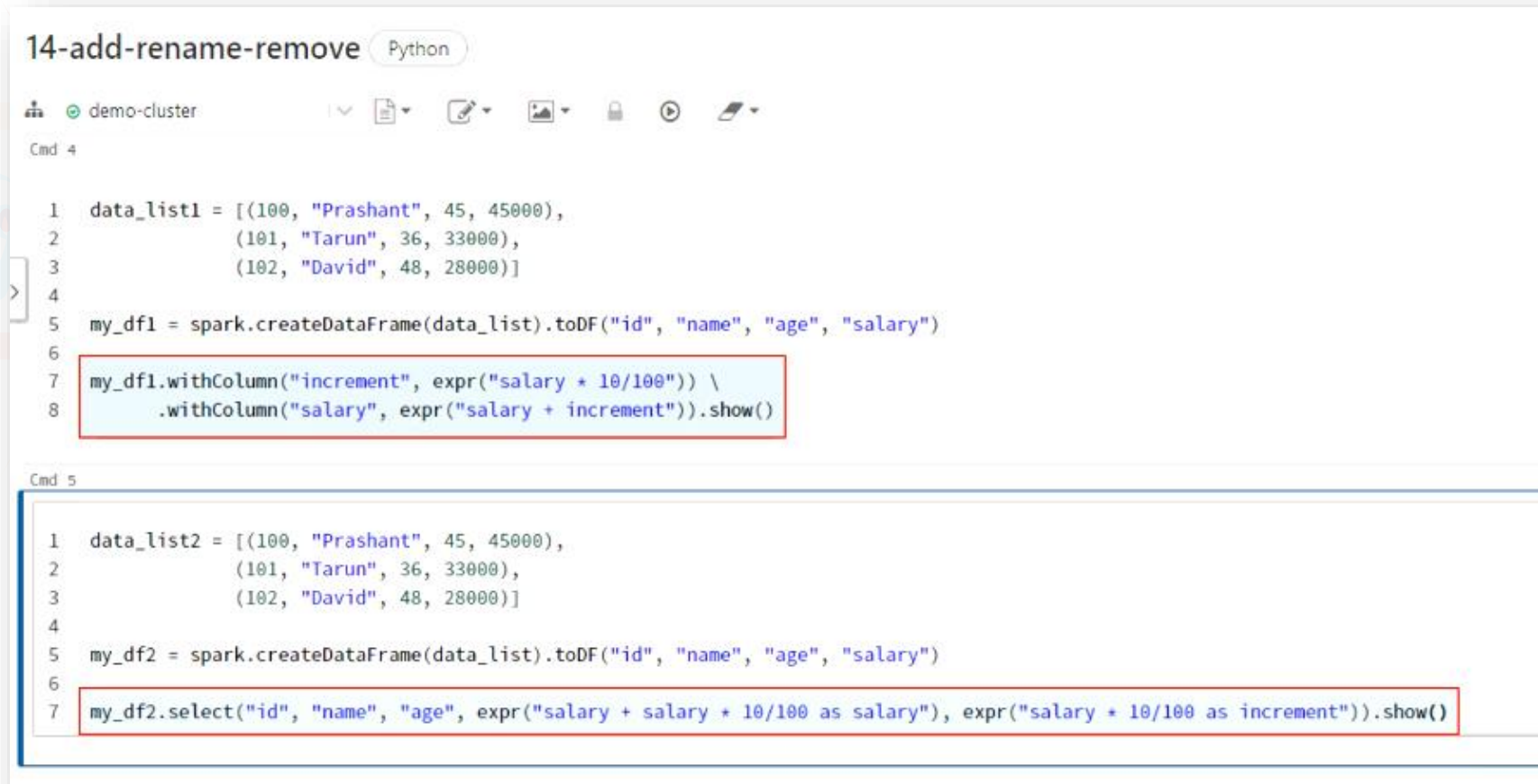
Now look at both codes and compare them.

I am creating data_list1 and then creating a dataframe using the same.

Then I am using the withColumn() method to add an increment column.

Then I am again using a withColumn() to change the salary column value.

Finally, I will show my_df1.



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell, labeled 'Cmd 4', contains Python code to create a list of tuples, convert it to a Spark DataFrame, and then use the `withColumn()` method to add an 'increment' column and update the 'salary' column. The second cell, labeled 'Cmd 5', contains Python code to create a list of tuples, convert it to a Spark DataFrame, and then use the `select()` method to create a new DataFrame with the same columns but with updated 'salary' and 'increment' values. Both code snippets are highlighted with red boxes.

```
14-add-rename-remove Python

demo-cluster

Cmd 4

1 data_list1 = [(100, "Prashant", 45, 45000),
2               (101, "Tarun", 36, 33000),
3               (102, "David", 48, 28000)]
4
5 my_df1 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")
6
7 my_df1.withColumn("increment", expr("salary * 10/100")) \
8         .withColumn("salary", expr("salary + increment")).show()

Cmd 5

1 data_list2 = [(100, "Prashant", 45, 45000),
2               (101, "Tarun", 36, 33000),
3               (102, "David", 48, 28000)]
4
5 my_df2 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")
6
7 my_df2.select("id", "name", "age", expr("salary + salary * 10/100 as salary"), expr("salary * 10/100 as increment")).show()
```

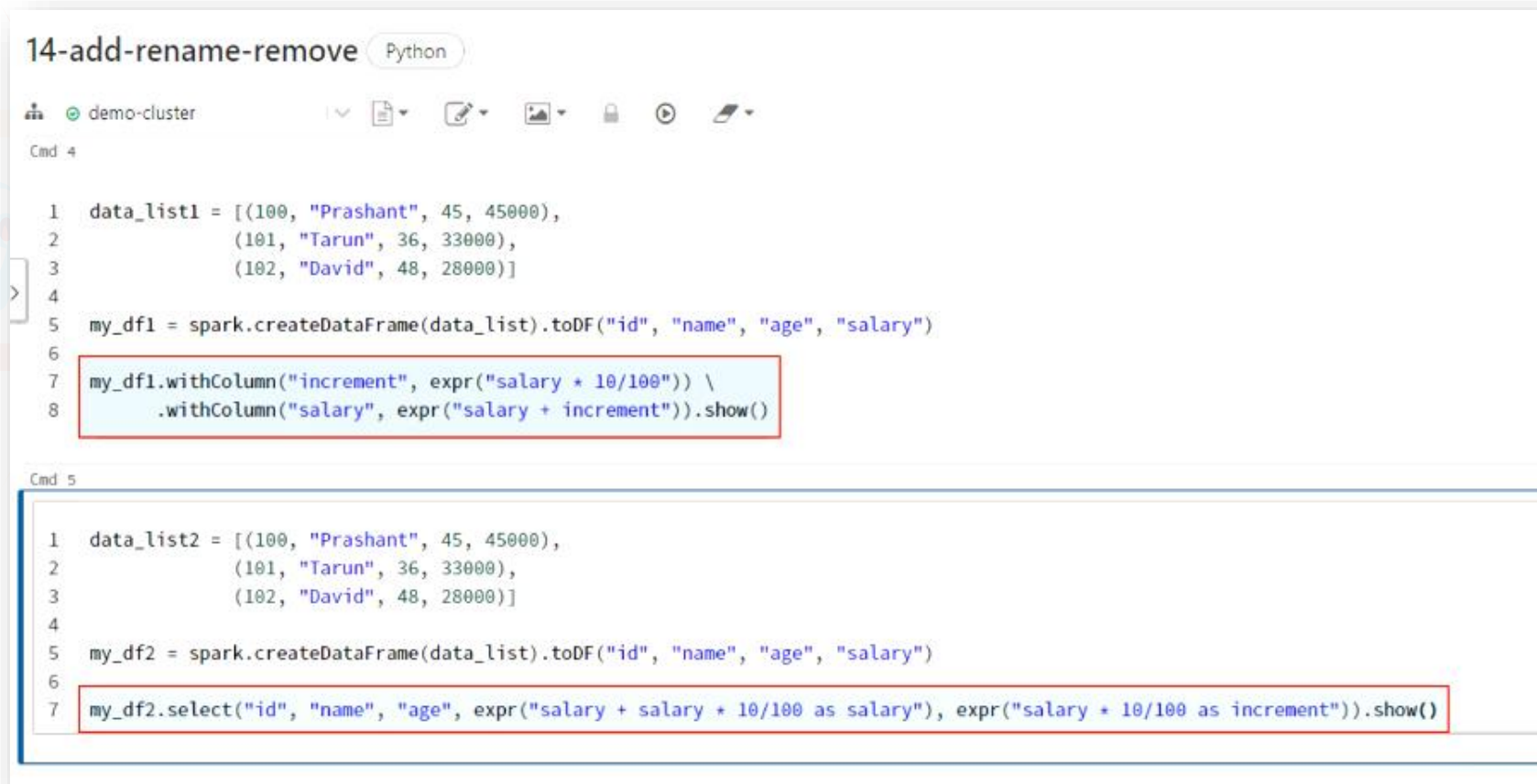
Now, look at the next cell.

I am doing the same thing here but taking a different approach. This time, I am creating data_list2, which is precisely the same as data_list1. Then I am using data_list2 to create my_df2. The my_df2 is also precisely the same as my_df1. Finally, I am using a select() method. The select method selects id, name, and age without any change. But I am using an expression to change the salary column value. Then I use another expression to add an increment column to my dataframe.

```
14-add-rename-remove Python
demo-cluster
Cmd 4
1 data_list1 = [(100, "Prashant", 45, 45000),
2               (101, "Tarun", 36, 33000),
3               (102, "David", 48, 28000)]
4
5 my_df1 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")
6
7 my_df1.withColumn("increment", expr("salary * 10/100")) \
8         .withColumn("salary", expr("salary + increment")).show()

Cmd 5
1 data_list2 = [(100, "Prashant", 45, 45000),
2               (101, "Tarun", 36, 33000),
3               (102, "David", 48, 28000)]
4
5 my_df2 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")
6
7 my_df2.select("id", "name", "age", expr("salary + salary * 10/100 as salary"), expr("salary * 10/100 as increment")).show()
```

Now compare the two withColumn() expressions in the first cell with the select() method in the second cell. Both the codes are doing the same thing. However, they take a different approach. The first approach is using a chain of two withColumn() methods. The second approach is using a single select() method. Coding the first approach is neat and convenient. But coding the second approach is a bit clumsy.



The screenshot shows a Jupyter Notebook interface with two code cells. The notebook title is '14-add-rename-remove' and the language is set to 'Python'. The environment is 'demo-cluster'. The first cell, labeled 'Cmd 4', contains code that creates a list of tuples, converts it to a Spark DataFrame, and then uses two chained withColumn() methods to calculate a 10% salary increment and update the salary column. The second cell, labeled 'Cmd 5', contains code that creates the same DataFrame and uses a single select() method to calculate the increment and select the original columns along with the new salary and increment columns. In both cells, the final result is shown using the show() method.

```
14-add-rename-remove Python  
demo-cluster  
Cmd 4  
1 data_list1 = [(100, "Prashant", 45, 45000),  
2               (101, "Tarun", 36, 33000),  
3               (102, "David", 48, 28000)]  
4  
5 my_df1 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")  
6  
7 my_df1.withColumn("increment", expr("salary * 10/100")) \  
8         .withColumn("salary", expr("salary + increment")).show()  
  
Cmd 5  
1 data_list2 = [(100, "Prashant", 45, 45000),  
2               (101, "Tarun", 36, 33000),  
3               (102, "David", 48, 28000)]  
4  
5 my_df2 = spark.createDataFrame(data_list).toDF("id", "name", "age", "salary")  
6  
7 my_df2.select("id", "name", "age", expr("salary + salary * 10/100 as salary"), expr("salary * 10/100 as increment")).show()
```

To check which one performs better. Run both the cells shown in the last slide, and check for the Spark UI, then go to the SQL tab. Open ID 8 and 7 in new tab.

ID	Description	Submitted	Duration	Job IDs
8	data_list2 = [(100, "Prashant", 45, 45000), ... + details	2022/05/09 09:22:22	0.7 s	[12] [13] [14]
7	data_list1 = [(100, "Prashant", 45, 45000), ... + details	2022/05/09 09:22:14	0.9 s	[9] [10] [11]
6	test_df2 = test_df1.withColumn("salary", expr("... + details	2022/05/09 09:19:08	0.9 s	[6] [7] [8]
5	from pyspark.sql.functions import expr test_df... + details	2022/05/09 09:17:13	1 s	[3] [4] [5]
4	data_list = [(100, "Prashant", 45, 45000), ... + details	2022/05/09 09:15:05	4 s	[0] [1] [2]
3	show tables in `default` + details	2022/05/09 09:12:37	33 ms	
2	show tables in `default` + details	2022/05/09 09:12:36	0.2 s	
1	show databases + details	2022/05/09 09:12:35	88 ms	
0	show databases	2022/05/09 09:12:32	2 s	

Come to the first one and check for the physical plan. The plan shows four steps, and we have two project operations in the SQL plan. So this one is the plan for the two consecutive withColumn() methods.

14-add-rename-remove - Databricks

Databricks Shell - SQL

Databricks Shell - Details for Que

Databricks Shell - Details for Que

+

community.cloud.databricks.com/sparkui/0509-085758-8uro796x/driver-7062008714697117811/SQL/execution/?id=7&o=946780288864704

Jobs

Stages

Storage

Environment

Executors

SQL

JDBC/ODBC Server

Structured Streaming

CollectLimit

▼Details

== Physical Plan ==

CollectLimit (4)

+ - * Project (3)

+ - * Project (2)

+ - * Scan ExistingRDD (1)

(1) Scan ExistingRDD [codegen id : 1]

Output [4]: [_1#87L, _2#88, _3#89L, _4#90L]

Arguments: [_1#87L, _2#88, _3#89L, _4#90L], MapPartitionsRDD[12] at applySchemaToPythonRDD at NativeMethodAccessorImpl.java:0, ExistingRDD, UnknownPartitioning(0)

(2) Project [codegen id : 1]

Output [5]: [_1#87L AS id#95L, _2#88 AS name#96, _3#89L AS age#97L, _4#90L AS salary#98L, (cast((_4#90L * 10) as double) / 100.0) AS increment#103]

Input [4]: [_1#87L, _2#88, _3#89L, _4#90L]

(3) Project [codegen id : 1]

Output [5]: [cast(id#95L as string) AS id#120, name#96, cast(age#97L as string) AS age#122, cast((cast(salary#98L as double) + increment#103) as string) AS salary#123, cast(increment#103 as string) AS increment#124]

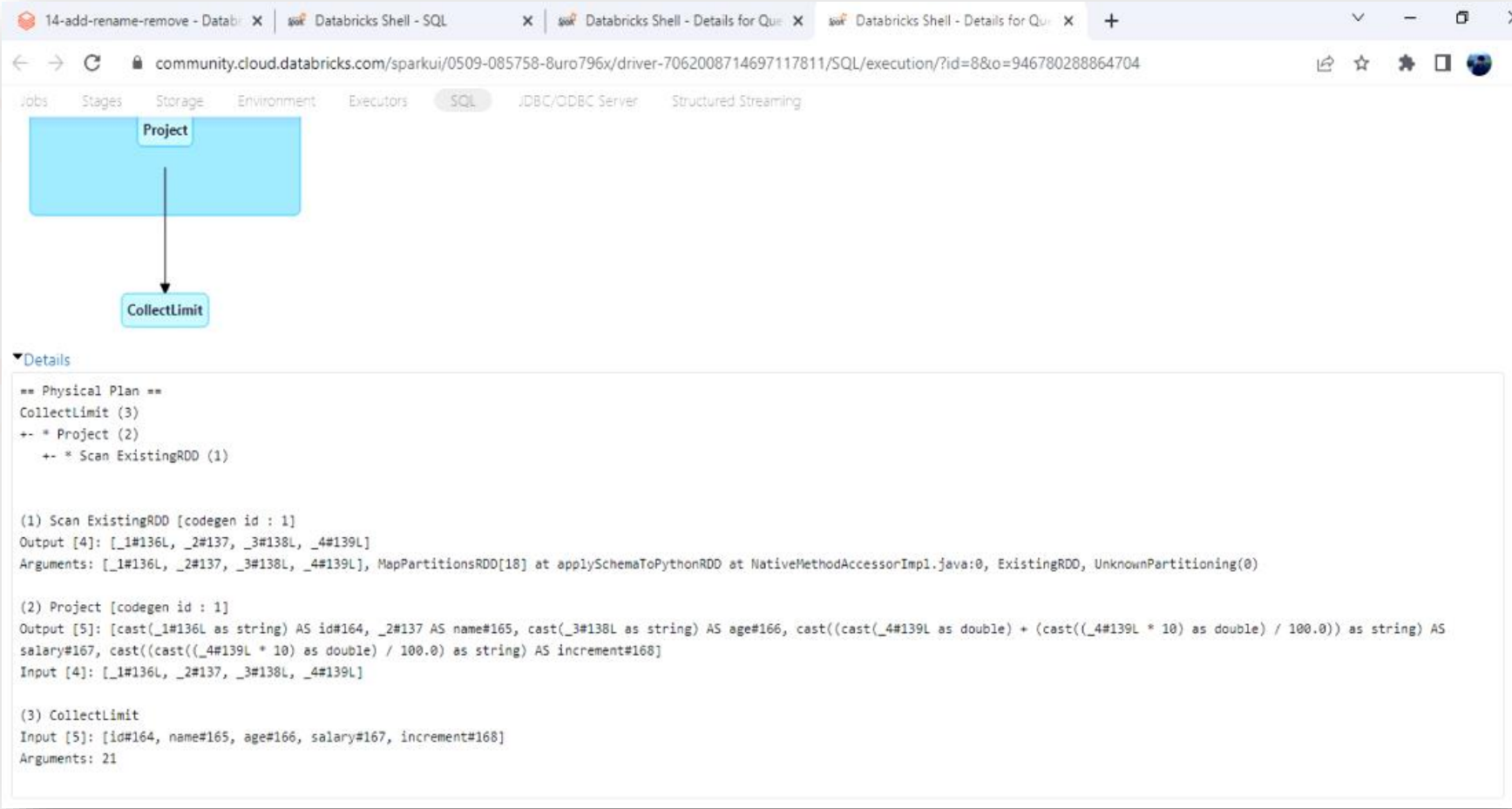
Input [5]: [id#95L, name#96, age#97L, salary#98L, increment#103]

(4) CollectLimit

Input [5]: [id#120, name#96, age#122, salary#123, increment#124]

Arguments: 21

Come to the second one and check for the physical plan. This plan shows only three steps and only one project operation. So the `select()` method worked faster. Every `withColumn()` adds one project operation in the SQL plan and slows down the SQL. So we recommend combining multiple `withColumn()` methods with a single `select()` method. This approach will cut down extra projections and speed up your SQL.



The screenshot shows the Databricks SQL execution plan for a query. The plan is visualized as a flow diagram with two main components: a 'Project' operation (represented by a blue box) and a 'CollectLimit' operation (represented by a light blue box). An arrow points from the 'Project' box to the 'CollectLimit' box, indicating the sequence of operations. Below the diagram, the 'Details' section provides a textual representation of the physical plan, showing the sequence of operations and their inputs/outputs.

Physical Plan ==

```
CollectLimit (3)
+- * Project (2)
   +- * Scan ExistingRDD (1)
```

(1) Scan ExistingRDD [codegen id : 1]
Output [4]: [_1#136L, _2#137, _3#138L, _4#139L]
Arguments: [_1#136L, _2#137, _3#138L, _4#139L], MapPartitionsRDD[18] at applySchemaToPythonRDD at NativeMethodAccessorImpl.java:0, ExistingRDD, UnknownPartitioning(0)

(2) Project [codegen id : 1]
Output [5]: [cast(_1#136L as string) AS id#164, _2#137 AS name#165, cast(_3#138L as string) AS age#166, cast((cast(_4#139L as double) + (cast((_4#139L * 10) as double) / 100.0)) as string) AS salary#167, cast((cast((_4#139L * 10) as double) / 100.0) as string) AS increment#168]
Input [4]: [_1#136L, _2#137, _3#138L, _4#139L]

(3) CollectLimit
Input [5]: [id#164, name#165, age#166, salary#167, increment#168]
Arguments: 21

Spark dataframe gives you the withColumnRenamed() method to rename an existing column. Here is an example shown below. I am using a chain of two withColumnRenamed() methods to rename two columns. You should note two things here.

- 1. The withColumnRenamed() is a no-op if the schema doesn't contain the given column name.
- 2. The withColumnRenamed() will also add one projection to your SQL plan. So chaining multiple withColumnRenamed() is not recommended.

14-add-rename-remove Python

demo-cluster

Command took 1.06 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:52:21 PM on demo-cluster

Cmd 6

```
1 test_df2.withColumnRenamed("increment", "salary_increment") \  
2     .withColumnRenamed("salary", "incremented_salary").show()
```

▶ (3) Spark Jobs

id	name	age	incremented_salary	salary_increment
100	Prashant	45	49500.0	4500.0
101	Tarun	36	36300.0	3300.0
102	David	48	30800.0	2800.0

Command took 0.62 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:59:02 PM on demo-cluster

Here are some facts about column naming.

1. Dataframe Column names are case insensitive.
2. You might be able to give a wizard name with spaces and use keywords without any problem.

But you will face problems when using those names in expressions.

In the example shown below, I renamed the salary column to a wizard name.

This new name has upper and lower case letters. We also have a keyword. The `as` is a keyword. I also have an arithmetic symbol in the column name. Such names are complete nonsense. But you can do it.

14-add-rename-remove Python

demo-cluster
Command took 0.57 seconds -- by prashant@scholarnest.com at 5/9/2022, 2:59:48 PM on demo-cluster

Cmd 8

```
1 test_df3 = test_df2.withColumnRenamed("salary", "SaLArY as INcrement + sALary")
2 display(test_df3)
```

▶ (3) Spark Jobs

	id	name	age	SaLArY as INcrement + sALary	increment
1	100	Prashant	45	49500	4500
2	101	Tarun	36	36300	3300
3	102	David	48	30800	2800

Showing all 3 rows.

Command took 0.51 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:01:36 PM on demo-cluster

So you won't have any problem creating crappy column names.

But you face a problem when you want to use them in an expression. In the code shown below, I want to subtract increment from the "salary as increment + salary." I am using all small case letters in the column name. And that's fine because Spark Dataframe column names are not case-sensitive. But we get an error.

Cmd 9

```
1 test_df3.withColumn("Salary as increment + salary", expr("salary as increment + salary - increment")).show()
```

⊞ **ParseException:**

mismatched input '+' expecting {'-'}(line 1, pos 20)

== SQL ==

salary as increment + salary - increment

-----^

Command took 0.20 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:03:51 PM on demo-cluster

You should not name your columns in such away.

The recommendation is to follow a standard naming convention and avoid giving such weird names.

But if you do, you can still use them in the expression. Enclose your column names in a pair of backtick characters. You will find the backtick character below the escape key on your keyboard. And you can see that it works.

```
> Cmd 9

1 test_df3.withColumn("Salary as increment + salary", expr("`salary as increment + salary` - increment")).show()

▶ (3) Spark Jobs

+-----+-----+-----+-----+
| id|   name|age|Salary as increment + salary|increment|
+-----+-----+-----+-----+
|100|Prashant| 45|          45000.0|    4500.0|
|101|  Tarun| 36|          33000.0|    3300.0|
|102|  David| 48|          28000.0|    2800.0|
+-----+-----+-----+-----+

Command took 0.95 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:04:52 PM on demo-cluster
```

So the point is straight.

Do not create column names with spaces, keywords, and symbols.

But you may still see such column names when you read data from external systems and sources.

Spark allows you to enclose such column names in a pair of backtick and use them in your expressions.

But you should rename such columns and give them a meaningful standard name.

To delete or remove a column from your Dataframe. Spark gives you a drop() method. Here in this example, I want to drop two columns from the test_df3 Dataframe. So I can use the drop() method and pass a comma-separated list of columns. It will drop all the given columns and give you a new Dataframe.

```
> Cmd 10

1 test_df3.drop("salary as increment + salary", "increment").show()

► (3) Spark Jobs

+----+-----+----+
| id|   name|age|
+----+-----+----+
|100|Prashant| 45|
|101|  Tarun| 36|
|102|  David| 48|
+----+-----+----+

Command took 0.74 seconds -- by prashant@scholarnest.com at 5/9/2022, 3:06:05 PM on demo-cluster
```



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com